LLMを用いたデータセット生成による ハードウェアトロイ検出における パープルチームフレームワーク

吉見 尚[1,a)]　戸川 望[1]

概要：近年，Internet of Things (IoT) 機器の普及に伴い，内部に搭載される集積回路の設計・製造工程におけるハードウェアトロイ (Hardware Trojan：HT) の挿入リスクが問題となっている．ゲートレベルネットリストを対象とした HT 検出では，グラフ学習を用いた HT 検出手法が提案されて高い検出性能を示しているが，公開されている HT 回路の種類と数が限られるため，多様な HT 構造に対する検出性能の確保が課題である．そこで本稿では，HT 検出器の解析結果に基づいて大規模言語モデルにより HT を生成し，生成 HT を用いた追加学習によって検出器を改善するパープルチームフレームワークを提案する．提案手法では，HT フリー回路および HT 回路に対する解析結果を HT 生成用プロンプトへ反映し，生成 HT を構文検査および論理等価性検証によって検証した後，学習データへ追加して検出器を再学習する．さらに，再学習後の解析結果を次の HT 生成へ反映することで，本フレームワークを反復的に適用する．評価実験の結果，既知の HT 回路のみを用いた場合に比較し，生成 HT を追加することで F 値が約 0.05 上回り，既存のグラフ学習ベースの HT 検出手法における結果を上回った．また，学習に用いていない生成 HT に対する評価により，未知の HT 構造に対する汎化性能を確認した．以上より，提案フレームワークがゲートレベル HT 検出性能の改善に有効であることを示した．

キーワード：ハードウェアトロイ, IoT セキュリティ, グラフ学習，LLM

A Collaborative Method for Improving Hardware Trojan Detector Performance Using LLMs and Graph Learning Models

Sho Yoshimi[1,a)]　Nozomu Togawa[1]

Abstract: In this paper, we propose a collaborative method for improving gate-level Hardware Trojan (HT) detection by combining LLM-based HT generation with graph learning models. Experimental results demonstrate that retraining the detector with generated HTs improves its detection performance, confirming the effectiveness of the proposed method.

Keywords: hardware-Trojan，IoT security, graph-learning，LLM

1. はじめに

近年, Internet of Things (IoT) 機器は, 家庭, 産業, 医療, 交通，社会インフラなど幅広い領域で利用されており，その機能や安全性は内部の集積回路 (Integrated Circuit: IC) に大きく依存する．一方，IC の設計・製造工程では，コスト削減や開発効率向上のために外部委託が広く行われており，多数の第三者が関与する半導体サプライチェーンにおいて，悪意のある回路であるハードウェアトロイ (Hardware Trojan: HT) が挿入されるリスクが指摘されている [1,2]．HT は，機密情報の漏洩，機能改ざん，性能劣化，機能停止，IC の破壊などを引き起こす回路であり，一般に，特定の入力や内部状態を検知するトリガ部と，悪意のある動作を実行するペイロード部から構成される [2,3]．HT は通常時には動作せず，小規模に実装されることが多いため，通

[1] 早稲田大学大学院基幹理工学研究科情報理工・情報通信専攻
Dept. Computer Science and Communications Engineering, Waseda University
[a)] sho.yoshimi@togawa.cs.waseda.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

常の機能テストによる検出は困難である．

HT 検出は，IC の設計段階および製造後の検査段階を対象として広く研究されている [2, 4]．本稿では，設計段階で得られるゲートレベルネットリストを対象とする．近年，ネットリストをグラフとして表現し，グラフ学習により HT を検出する手法が提案され，高い有効性が示されている [5–11]．しかし，Trust-HUB [12] や TRIT-TC，TRIT-TS [13] などの公開ベンチマークに収録された HT 回路は数や種類が限られている．そのため，学習データが特定の回路構造や挿入方法に偏り，未知の HT に対して十分な検出性能を得られない可能性がある．上記課題への解決策として，大規模言語モデル (LLM) による HT 生成が考えられる．RTL を対象とした LLM ベースの HT 生成手法 [14, 15] は，多様な HT を自動生成できるが，ゲートレベル HT 検出へ適用するには論理合成と前処理を要する．ゲートレベルネットリストを対象とした HT 生成手法 [16, 17] も提案されているが，LLM を用いた HT 生成は行われていない．

本稿では，LLM による HT 生成とグラフ学習型 HT 検出器の再学習を反復する，ゲートレベル HT 検出向けパープルチームフレームワークを提案する．LLM が生成した HT を検証後に学習データへ追加し，強化された検出器の解析結果を次の HT 生成へ反映することで，生成側と検出側を相互に強化する．

本稿の貢献を以下に示す．

(1) LLM による HT 生成と，生成 HT を用いたグラフ学習型 HT 検出器の再学習を反復するパープルチームフレームワークを提案した．

(2) ゲートレベルネットリストを対象とした LLM による HT 生成手法を構築し，構文検査および論理等価性検証を通過する HT 回路を生成した．また，生成 HT が一定の検出回避性能を有し，加えて HT 検出器の性能を向上させるための学習データとして有効であることを示した．

(3) 生成 HT を用いた再学習により HT 識別性能を向上させ，最大 F 値 0.9779 を達成した．また，既存のゲートレベル HT 検出手法の報告値を上回る F 値を得た．

2. 背景

2.1 関連研究

本稿では，IC 設計段階のゲートレベルネットリストを対象とした HT 検出に焦点を当てる．IC の開発工程は，仕様策定，RTL 設計，論理合成，物理設計，レイアウト生成，製造の順に進み，各工程で HT が挿入される可能性がある．設計段階で HT を検出することで，製造後の全デバイスを検査する場合に比べて，検査時間とコストを削減できる．また，RTL 段階で挿入された HT も論理合成後のネットリストに反映されるため，ゲートレベルネットリストを対象とすることで，RTL 段階およびゲートレベル段階で挿入された HT を統一的に検出できる．

ゲートレベルネットリストを対象とした HT 検出手法は，初期には静的解析に基づく手法 [18] や，HT のトリガ条件を活性化するテスト生成手法 [19]，形式的解析手法 [20]，および機械学習に基づく手法 [21] などが提案されている．機械学習に基づく手法では，ゲートレベルネットリストから抽出した特徴量を用いて，各ゲートまたはネットが HT として挿入されたものかを識別する手法が数多く提案されている [22–24]．近年では，ゲートレベルネットリストをグラフとして表現し，ゲートの特徴と回路の接続構造を同時に学習するグラフ学習に基づく手法が提案されており，高い有効性を示している [5–11]．しかし，ゲートレベルネットリストを対象とした機械学習ベースの HT 検出手法では，Trust-HUB [12] や TRIT-TC，TRIT-TS [13] などの公開ベンチマークを用いて学習・評価を行うことが一般的である．一方，公開されている HT 回路の種類と数は限られているため，学習データが HT の構造的多様性を十分に網羅しているとは言い難く，多様な HT 回路に対する汎化性能の評価も限定的である．

上記課題への解決策の 1 つとして，LLM を用いた HT 生成手法が考えられる．RTL では，LLM を用いた HT 生成手法が提案されており [14, 15]，従来の手動設計やルールベース手法に代わり，自動的に多様な HT 回路を生成可能である点で注目されている．しかし，RTL で生成された HT 回路をゲートレベルネットリストを対象とした HT 検出手法に適用するには，RTL を論理合成してゲートレベルネットリストへ変換し，学習できるように前処理をする必要がある．TrojanGYM [14] では，HT の検出回避性能を向上させるために検出器の出力のフィードバックを用いているが，その反復は HT 生成に閉じている．また，ゲートレベルネットリストを対象とした HT 生成手法 [16, 17] も提案されているが，LLM を用いた HT 生成は行われていない．TrojanForge [16] では，強化学習による HT 検出回避を目的とした HT 生成が行われている．

以上の議論より，関連研究の課題を以下に整理する．

課題 1) ゲートレベルネットリストを対象としたグラフ学習型 HT 検出手法は高い有効性を示しているが，公開 HT 回路が限定されているため，学習データに十分かつ多様な HT が含まれているとは限らず，未知の HT 回路に対する汎化性能も十分に評価されていない．

課題 2) 課題 1 の解決策として，LLM を用いた HT 生成手法が提案され，検出回避性能も評価されている．しかし，ゲートレベルネットリストを対象とした手法は提案されていない．

提案フレームワークでは，上記 2 つの課題を解決し，より汎用的で高検出性能な HT 検出器の作成を目的とする．

2.2 ゲートレベルネットリストを対象にした機械学習を用いた HT 検出

ゲートレベルネットリストは，論理ゲートとそれらを接続するネットにより，回路の論理構造と接続関係を表

© 2026 Information Processing Society of Japan

表 1 評価指標.

指標	定義
TP	正しく識別されたトロイゲート数
TN	正しく識別されたノーマルゲート数
FP	トロイゲートと誤識別されたノーマルゲート数
FN	ノーマルゲートと誤識別されたトロイゲート数
TPR	トロイゲートのうち，正しく識別されたゲートの割合：TP/(TP + FN)
Precision	トロイゲートと識別されたゲートのうち，真にトロイゲートである割合：TP/(TP + FP)
F 値	TPR と Precision の調和平均：2/(1/TPR + 1/Precision)
TNR	ノーマルゲートのうち，正しく識別されたゲートの割合：TN/(TN + FP)
Accuracy	すべてのゲートのうち，正しく識別されたゲートの割合：(TP + TN)/(TP + TN + FP + FN)

現した設計データである．ゲートレベルネットリストを対象とした機械学習ベースの HT 検出では，HT 検出器が各ゲートまたは各ネットが，HT として挿入されたかを識別する．本稿ではゲート単位の識別を行い，HT 検出器による識別結果は，表 1 に示す True Positive (TP)，True Negative (TN)，False Positive (FP)，および False Negative (FN) の 4 種類に分類される．4 つの分類結果に基づいて，TPR，Precision，F 値，TNR，および Accuracy を算出する．HT 検出では，HT の見逃しを表す TPR と誤検出に関係する Precision，および両者の調和平均である F 値が重視される．

ゲートレベルネットリストを対象とした機械学習ベースの HT 検出の流れを，本稿で使用する GraphGPS を用いた HT 検出手法 [11] を例に説明する．HT 検出の処理は，データセット作成段階，学習段階，および評価段階の 3 段階から構成される．データセット作成段階では，ゲートレベルネットリストをグラフに変換し，特徴量追加やエンコーディングといった前処理を行う．作成したグラフは，グラフ単位で学習用データセットと評価用データセットに分割する．次に学習段階では，学習用データセットを用いて GraphGPS で学習することで，各ゲートがトロイかを識別する HT 検出器を作成する．評価段階では，学習段階で作成した HT 検出器を使用して評価用データセットに対する HT 識別精度を評価する．詳細な評価方法と各種パラメータは 3 章に示す．

3. 提案フレームワーク

本稿では，LLM による検出されにくい HT の生成と，生成 HT を用いた検出器の再学習を反復し，生成側と検出側を相互に強化するゲートレベル HT 検出向けパープルチームフレームワークを提案する．図 1 に示すように，本フレームワークは，HT を生成するレッドチーム，検出器を強化するブルーチーム，および両者を中継するオーケストレータから構成される．

3.1 パープルチームフレームワークの概要

提案フレームワークの流れを以下に示す．

手順 1. 既知データセット構築 (ブルーチーム)：
既知の HT 回路を学習・評価に使用できるように前処理を行うことで，既知データセットを構築する．本稿では，表 2 に示す Trust-HUB に公開されている 17 種類の HT 回路を既知の HT 回路とする．すべての HT 回路をグラフに変換し，学習データと評価データに分割する．文献 [5] にならい，HT 検出器の評価に 1 個抜き交差検証を使用する．#1〜#17 を 1 個ずつ評価データとし，それぞれ残りの 16 個を学習データとする．例えば，#1 の回路を評価する場合は#2〜#17 のグラフを学習データとし，#1 を評価データとする．

表 2 実験に使用するネットリストのデータ (Trust-HUB).

#	ネットリスト名	ノーマルゲート数	トロイゲート数
1	RS232-T1000	289	13
2	RS232-T1100	293	11
3	RS232-T1200	296	10
4	RS232-T1300	290	9
5	RS232-T1400	290	12
6	RS232-T1500	291	13
7	RS232-T1600	290	13
8	s15850-T100	2397	27
9	s35932-T100	5967	15
10	s35932-T200	5962	15
11	s35932-T300	5965	36
12	s38417-T100	5656	12
13	s38417-T200	5656	15
14	s38417-T300	5659	44
15	s38584-T100	7064	9
16	s38584-T200	7064	83
17	s38584-T300	7064	731

表 3 学習およびモデルの主要パラメータ設定.

項目	設定値
MPNN 層	GatedGCN
Transformer 層	Transformer
GPS 層の層数 L	4
隠れ層次元 d	128
Attention head 数	4
ドロップアウト	0.168
最適化手法	AdamW
学習率	2.52×10^{-4}
Weight decay	6.40×10^{-4}
エポック数	1000
バッチサイズ	2
勾配累積	8
スケジューラ	Cosine annealing with warmup
ウォームアップ	50 エポック
損失関数	Weighted Cross Entropy
サンプリング	SAINT (random walk)
LapPE	$K = 4$, $d = 4$, DeepSet (3 層)
RWSE	$k =$ 1–16, $d = 16$, BatchNorm + Linear

表 4 HT を挿入するフリー回路.

#	ネットリスト名	ノーマルゲート数	HT ゲート数
1	RS232_free	284	0
2	s15850_free	2396	0
3	s35932_free	5965	0
4	s38417_free	5656	0
5	s38584_free	7063	0

手順 2. HT 検出器作成 (ブルーチーム)：
学習データを使用して，機械学習を用いた HT 検出器を作成する．使用する機械学習アーキテクチャは GraphGPS [25] を使用し，各種パラメータを表 3 に示す．

手順 3.1 HT 検出器評価 (オーケストレータ)：
手順 2 で作成した HT 検出器の評価を行う．1 個抜き

© 2026 Information Processing Society of Japan

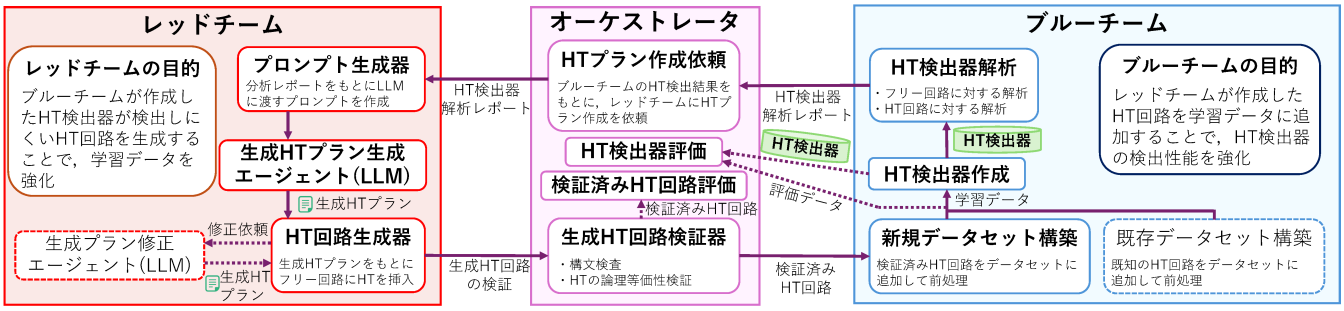


図 1　提案フレームワークの概要.

交差検証をするために 17 種類の HT 検出器を作成する．表 2 に示す HT 回路 1 つずつに対する HT 識別精度を評価する．例えば，#17 を評価データとして除き #1〜#16 のグラフを学習した HT 検出器は，#17 のグラフの各ノードが HT かどうかを識別し，その識別精度を評価する．

手順 3.2 HT 検出器解析 (ブルーチーム)：
手順 2 で作成した HT 検出器を使用して，フリー回路に対する解析と HT 回路に対する解析を行う．HT を挿入する対象である表 4 に示す 5 種類のフリー回路に対して解析する．解析内容の詳細は， 3.2 節で説明する．

手順 4 プロンプト生成器 (レッドチーム)：
手順 3.2 で得られた HT 検出器の解析結果に基づき，HT 生成用のプロンプトを作成する．表 5 に，LLM に与えるプロンプトの構成と各項目の説明を示す．安全上の観点から詳細なプロンプトの記載は避けるが，フリー回路に対する解析によって得られた $\mathrm{TargetScore}_v$ の値が大きいゲート v をターゲットゲートとして選択し，`TARGET CONTEXT` にはターゲットゲートの情報を，`STRUCTURAL CONTEXT` には HT 生成に使用する周辺ゲートおよび接続関係の情報を与える．また，HT 回路に対する解析結果を `STEALTH CAPABILITY` に反映し，生成時に推奨または回避するゲート種別を指定する．`STEALTH CAPABILITY` と `TOPOLOGY EVIDENCE` をプロンプトに加えることで，既存の HT の構造とは異なる構造の HT を生成することを誘導する．出力形式には，生成する HT のゲートレベル構造情報，トリガの活性化条件，およびペイロードの動作を含む JSON 形式を指定する．これにより，LLM の出力を機械的に解釈し，フリー回路への HT 挿入処理に利用できる．さらに，回路全体ではなくターゲットゲートとその周辺情報のみを与えることで，入力トークン数および API 利用コストを削減する．上記のプロンプト構成にすることで，ブルーチームが作成した HT 検出器が検出しにくい HT 回路を生成することを目指す．

手順 5 生成 HT プラン作成エージェント (レッドチーム)：
手順 4 で作成したプロンプトを使用して，LLM による HT 生成を行う．本稿では，LLM として gemini-3-flash-preview を使用する．

表 5　LLM に与えるプロンプトの構成.

項目	説明
`SYSTEM`	LLM の役割と出力形式を指定する.
`STEALTH CAPABILITY`	推奨・回避するゲート種を指定する.
`TARGET CONTEXT`	HT の挿入対象となるゲートを指定する.
`CONSTRAINTS`	トリガとペイロードの生成条件を指定する.
`PAYLOAD LOGIC SPEC`	ペイロードが満たすべき動作を指定する.
`CELL LIBRARY`	使用可能なゲート種を指定する.
`STRUCTURAL CONTEXT`	対象ゲートの周辺構造と候補信号を与える.
`TOPOLOGY EVIDENCE`	過去に生成された HT の構造を与える.
`NOVELTY CONTRACT`	構造的に異なる HT の生成を指示する.
`OUTPUT SPEC`	出力する JSON の形式を指定する.
`PIN ROLE RULE`	MUX の各端子に接続する信号を指定する.

手順 6 HT 回路生成器 (レッドチーム)：
手順 5 で作成した生成 HT プランをもとに，機械的にフリー回路に HT を挿入することで HT 回路を生成する．LLM が意図しない形式で出力したときは，生成プラン修正エージェントに修正依頼をすることも可能である．

手順 7 生成 HT 回路検証器 (オーケストレータ)：
生成 HT 検証器で，レッドチームが生成した HT が構文的，機能的に正しいかを検証する．検証方法の詳細は 3.3 節で説明する．

手順 8.1 検証済み HT 回路評価 (オーケストレータ)：
手順 7 で検証を通過した HT 回路を対象に，既存の HT 検出器がどの程度検出できるかを評価する．

手順 8.2 新規データセット構築 (ブルーチーム)：
新たに生成した検証済み HT 回路を HT 検出器の学習に使用できるように前処理を行うことで，新規データセットを構築する．新規データセットを HT 検出器の学習に追加することで，HT 検出器がより多様な HT を学習することができ，HT 検出器の検出性能の強化と汎化性能の強化を目指す．

以降，手順 2 から手順 8 を繰り返すことで，LLM による HT 検出器が検出しづらい HT 回路の生成と，HT 検出器の強化を繰り返し実行できる．

3.2　HT 検出器の性能解析 (手順 3.2)

フリー回路に対する解析では，各ゲート v に対して以下

© 2026 Information Processing Society of Japan

に示す $\mathrm{TargetScore}_v$ を計算する．

$$\mathrm{TargetScore}_v = \frac{1}{3}\Big(\mathrm{Importance}_v + (1-\mathrm{FPP}_v) + (1-\mathrm{Saliency}_v)\Big) \quad (1)$$

$\mathrm{Importance}_v$ は回路構造上の重要度を表し，重要なノードとの接続を評価する PageRank (PR) [26] と，信号伝搬経路の中継点としての重要性を評価する Betweenness Centrality (BC) [27] を用いて算出する．両指標を回路内で 0–1 に正規化し，次式により統合する．

$$\mathrm{Importance}_v = \frac{1}{2}PR_{\mathrm{norm}}(v) + \frac{1}{2}BC_{\mathrm{norm}}(v). \quad (2)$$

FPP_v は，フリー回路中のゲート v を HT と予測する確率であり，$(1-\mathrm{FPP}_v)$ によって HT と誤識別されにくいゲートを高く評価する．$\mathrm{Saliency}_v$ は，モデル出力 y に対するゲート v の各入力特徴量 $x_{v,k}$ の勾配絶対値和を Min-Max 正規化した値である．

$$\mathrm{Saliency}_v = \mathrm{MinMax}_{u\in V}\left(\sum_k \left|\frac{\partial y}{\partial x_{v,k}}\right|\right) \quad (3)$$

$(1-\mathrm{Saliency_v})$ により，モデルが特徴量変化に対して鈍感なゲートを高く評価する．手順 4 では，$\mathrm{TargetScore}_v$ が高いゲートを HT のターゲット候補とする．

HT 回路に対する解析では，学習に使用したすべての HT 回路を対象として，ゲート種別ごとの平均 TPR を算出する．平均 TPR が高いゲート種を検出されやすい種別，低いゲート種を検出されにくい種別として評価し，HT 生成に反映する．

3.3 生成 HT 検証器 (手順 7)

生成 HT 回路に対し，Icarus Verilog [28] による構文検査と，Cadence Conformal Equivalence Checker (Conformal EC) [29] による論理等価性検証を行う．構文検査では，構文エラー，未定義セル，およびポート接続の不整合がないことを確認する．次に，論理等価性検証では生成プランに記載されたトリガ入力の全組合せについて，フリー回路と生成 HT 回路を比較し，指定条件以外では論理的に等価であり，指定条件でのみ非等価となることを確認する．さらに，トリガ活性時の生成 HT 回路を期待ペイロードモデルと比較し，指定されたペイロード動作と論理的に等価であることを検証する．例えば bit-flip 型では，ターゲットゲートの出力を反転したモデルを期待ペイロードモデルとする．

以上の検証をすべて通過した回路のみを検証済み HT 回路として採用する．

4. 評価実験

4.1 実験環境

本稿では，メモリ 1.5 TB, Xeon Gold 6230R CPU, Tesla V100S-PCIE-32GB GPU を搭載した計算機で評価実験を行った．また，ソフトウェア環境として，Python 3.8.10, PyTorch 2.4.1，および PyTorch Geometric 2.6.1 を使用した．

HT 識別性能は， 2.2 節で説明したように，表 1 に示す評価指標で評価する．既知の HT 回路として表 2 に示す Trust-Hub に公開されている 17 種類の HT 回路を使用し，HT を挿入するフリー回路として表 4 に示す 5 種類のフリー回路を使用する．使用する HT 回路は，表 4 のいずれかに HT が挿入された HT 回路である．

表 6 生成世代間における追加 HT の構造比較結果.

対象 HT 回路	正規化 GED	WL 類似度
gen2 → gen1	0.250 (0.183–0.315)	0.412 (0.390–0.429)
gen3 → gen1	0.212（0.146–0.250）	0.429（0.400–0.440）
gen3 → gen2	0.155 (0.086–0.232)	0.502 (0.443–0.605)
gen2′ → gen2	0.059（0.017–0.086）	0.864（0.846–0.880）

4.2 生成 HT の類似性評価

LLM による HT 生成にあたり，トリガには 4–8 入力の組合せ回路を指定し，ペイロードには bit-flip，stuck-at-0/1 (0/1 に信号固定)，mux (信号すり替え)，delay (出力遅延) の 5 種類を用いる．1 回目の生成 (3.1 節の手順 1〜手順 8 を 1 回実行，gen1 とする) では，各フリー回路の $\mathrm{TargetScore}_v$ 上位 5 ゲートを対象に 125 件の HT 回路を生成し，そのうち検証を通過した 106 件の HT 回路を採用した．2 回目の生成 (gen 2) と 3 回目の生成 (gen 3) では，上位 5 ゲートからランダムに 2 ゲートを選択して各 50 件の HT 回路を生成し，それぞれ 37 件および 49 件の HT 回路を採用した．また，構造類似性の比較対象として，gen 2 の HT 回路に対して手動で最小限の変更を加えた HT 回路 (gen 2′) を作成した．

生成世代間の構造差を評価するため，同一のフリー回路およびペイロードに属する追加 HT 部分のグラフを比較する．正規化グラフ編集距離 (Graph Edit Distance: GED) は，ノードおよびエッジの編集コストをグラフ規模で正規化した値であり，0 に近いほど類似する [30]．Weisfeiler-Lehman (WL) 類似度は，各ノードラベルと近傍構造から得られる特徴ベクトル間のコサイン類似度であり，1 に近いほど類似する [31].

表 6 に比較結果を示す．生成世代間では，gen 2′ との比較より正規化 GED が高く，WL 類似度が低いことから，単純なゲート置換や追加を超える構造変化が確認された．また，世代ごとの TargetScore の再計算により，gen 2 および gen 3 ではそれぞれ 5 個の新たなターゲットゲートが選択された．

4.3 HT 検出器の HT 識別性能評価

HT 検出器の HT 識別性能評価として，既知データセット (og) を用いて作成した HT 検出器の HT 識別性能 (初期 HT 識別結果) と，LLM が生成した HT 回路を新規データセットとして追加して作成した HT 検出器の HT 識別性能を評価する．評価には，17 個の既知 HT 回路を対象とし

© 2026 Information Processing Society of Japan

表 7 学習データの構成ごとの HT 識別性能.

学習データ	F 値	TPR（%）	Precision（%）	TNR（%）	Accuracy（%）
og	0.9226	90.67	98.30	99.96	99.89
og+gen1	0.9514	96.03	95.24	99.95	99.88
og+gen2	0.9779	96.62	99.54	100.00	99.91
og+gen1+gen2	0.9684	97.86	96.15	99.91	99.86
og+gen3	0.9727	96.62	98.45	99.96	99.88
og+gen1+gen2+gen3	0.9500	96.29	94.85	99.95	99.88

表 8 各ネットリストに対する HT 識別結果の変化.

#	ネットリスト	TN	FP	FN	TP	F 値	TPR（%）	Precision（%）
1	RS232-T1000	289	0	0	13	1.0000	100.00	100.00
2	RS232-T1100	293	0	0	11	1.0000	100.00	100.00
3	RS232-T1200	296	0	0	10	1.0000	100.00	100.00
4	RS232-T1300	290	0	0	9	1.0000	100.00	100.00
5	RS232-T1400	290	0	0	12	1.0000	100.00	100.00
6	RS232-T1500	291	0	0	13	1.0000	100.00	100.00
7	RS232-T1600	288 → 290	2 → 0	3 → 4	10 → 9	0.8000 → 0.8182	76.92 → 69.23	83.33 → 100.00
8	s15850-T100	2396 → 2397	1 → 0	0	27	0.9818 → 1.0000	100.00	96.43 → 100.00
9	s35932-T100	5967	0	0	15	1.0000	100.00	100.00
10	s35932-T200	5961	1	4 → 0	11 → 15	0.8148 → 0.9677	73.33 → 100.00	91.67 → 93.75
11	s35932-T300	5965	0	0	36	1.0000	100.00	100.00
12	s38417-T100	5656	0	0	12	1.0000	100.00	100.00
13	s38417-T200	5656	0	3 → 4	12 → 11	0.8889 → 0.8462	80.00 → 73.33	100.00
14	s38417-T300	5659	0	0	44	1.0000	100.00	100.00
15	s38584-T100	7064	0	8 → 0	1 → 9	0.2000 → 1.0000	11.11 → 100.00	100.00
16	s38584-T200	7064 → 7063	0 → 1	0	83	1.0000 → 0.9940	100.00	100.00 → 98.81
17	s38584-T300	7063 → 7061	1 → 3	0	731	0.9993 → 0.9980	100.00	99.86 → 99.59
–	平均	–	–	–	–	0.9226 → 0.9779	90.67 → 96.62	98.30 → 99.54

た 1 個抜き交差検証を用いる．生成 HT を n 個追加する場合，各評価回路を除く 16 個の既知 HT と n 個の生成 HT を用いて学習し，生成 HT は評価データには含めない．これにより，生成 HT の追加による既知 HT に対する識別性能の変化を評価する．初期モデルと生成 HT を追加して再学習したモデルを比較して性能が向上した場合，生成 HT の追加が多様な HT 特徴の学習と検出性能の向上に有効であると評価する．

表 7 に，各世代における学習データの構成ごとの HT 識別性能を示す．生成 HT を追加したすべての構成で，og のみの場合より F 値および TPR が向上した．特に，og+gen2 では最大 F 値 0.9779 を達成した．一方，gen3 や全世代の HT を追加してもさらなる向上は得られず，本評価では og+gen2 が最も有効な学習データ構成であった．これは，より多様な HT を識別するように学習したことで検出対象の範囲が広がる一方，特定の HT に対する識別性能がわずかに低下した可能性がある．

次に，表 8 に，初期モデルと og+gen2 で学習したモデルにおける各ネットリストの HT 識別結果を示す．表中の矢印は初期モデルから og+gen2 モデルへの変化を示し，単一の値は両モデルで同一であることを示す．s38584-T100 では F 値が 0.2000 から 1.0000 に大幅に向上した．一部のネットリストでは性能低下も見られたが，全体として HT 識別性能の向上を確認した．

4.4 生成 HT に対する HT 識別性能の評価と HT 検出器の汎化性能評価

表 9 に，学習データごとの各世代 HT に対する識別性能を示す．og に対する結果は 1 個抜き交差検証により算出し，生成 HT については各モデルの学習データに含まれない世代のみを評価した．og のみで学習したモデルの TPR は，gen 1，gen 2，gen 3 に対してそれぞれ 13.5%，7.93%，6.88%であり，既存の学習データでは検出されにくい HT を生成できていることが分かる．また，次世代の未知 HT に対する TPR は，og+gen 1 で学習したモデルの gen 2 評価で 89.8%，og+gen 1+gen 2 で学習したモデルの gen 3 評価で 94.7%となった．これらは各モデルの og に対する TPR と比べてそれぞれ 6.2 ポイントおよび 3.2 ポイントの低下であり，高い検出率ではあるものの HT 識別精度の低下につながる HT 生成結果であることがわかる．

表 9 学習データごとの各世代 HT に対する HT 識別性能.
各セルは F 値 / TPR（%）を示す．gn は gen n を表す.

学習データ	og	g1	g2	g3
og	0.923 / 90.7	0.193 / 13.5	0.137 / 7.93	0.116 / 6.88
og+g1	0.951 / 96.0	–	0.929 / 89.8	0.874 / 82.8
og+g2	0.978 / 96.6	0.885 / 86.4	–	0.971 / 95.7
og+g1+g2	0.968 / 97.9	–	–	0.970 / 94.7
og+g3	0.973 / 96.6	0.884 / 86.7	0.947 / 91.8	–
og+g1+g2+g3	0.950 / 96.3	–	–	–

一方，og のみで学習したモデルと比較すると，生成 HT を用いて再学習したモデルは，学習データに含まれない次世代および前世代の HT に対しても大幅に高い TPR を示した．これは，多様な構造的特徴を持つ生成 HT を学習データに含めることで，特定の世代に依存しない HT の特徴を学習できたためと考えられる．したがって，生成 HT を用いた再学習により，未知の HT に対する HT 検出器の汎化性能が向上したことが確認できる．

© 2026 Information Processing Society of Japan

表 10　他手法との HT 識別精度比較.

手法	評価ネットリスト数	F 値	TPR (%)	Precision (%)	TNR (%)	Accuracy (%)
提案手法	17	**0.9779**	**96.62**	**99.54**	**100.00**	**99.91**
NHTD-GL [5] (2023)	24	0.9211	89.04	97.75	99.95	99.79
	17	0.9030	86.54	97.40	99.94	99.70
GATrojan [6] (2024)	17	0.9217	96.04	90.05	99.42	-
GraphSAGE-LSTM [7] (2025)	17	0.9095	88.06	-	99.96	-
GNN4Gate [8] (2022)	17	0.8161	87.14	82.79	99.73	99.61
[9] (2025)	132	0.860	84.1	91.2	-	-
GREAT [10] (2025)	24	0.9618	94.58	99.2	99.99	**99.9**
	17	–	94.61	–	**100.00**	–

4.5　ゲートレベルネットリストを対象としたグラフ学習を用いた HT 検出手法との比較

表 10 に，ゲートレベルネットリストを対象とする既存のグラフ学習を用いた HT 検出手法との比較結果を示す．17 ネットリストを用いた比較では，提案手法は最大で F 値 0.9779，TPR 96.62%，Precision 99.54%，TNR 100%，Accuracy 99.91%を達成し，既存手法の報告値を上回る HT 識別性能を示した．また，TPR と Precision の双方で高い値を維持しており，HT の見逃しと誤検出を抑えた識別が可能であることが分かる．

4.6　考察

本稿では， 2.1 節で示した 2 つの課題に対し，LLM による HT 生成とグラフ学習型 HT 検出器の強化を反復するパープルチームフレームワークを提案した．課題 1 に対して，生成 HT を学習データに追加することで，最大 F 値 0.9779 を達成し，既存手法の報告値を上回る性能が得られた．また，学習データに含まれない前世代および次世代の HT にも高い TPR を示したことから，多様な HT の学習によって未知 HT に対する汎化性能が向上したと考えられる．課題 2 に対して，ゲートレベルネットリストを対象とした LLM による HT 生成手法を構築した．生成 HT は構文検査および論理等価性検証を通過し，世代ごとに異なる構造とターゲットを持つとともに，既知 HT のみで学習した検出器に対して低い TPR を示した．さらに，生成 HT を再学習に用いることで検出器の性能向上にもつながったことから，HT 生成と検出器強化を相互に反復する提案フレームワークの有効性が示された．

今後の課題として第一に，より大規模な回路への適用が挙げられる．ゲートレベルネットリストを対象とした HT 検出は実用化も進められており [32]，実環境では特定の回路を対象とした検査が想定されるため，本評価のように対象回路に対する検出性能を向上させることには意義がある．一方，近年の IC の大規模化を踏まえ，より大きなネットリストにおける検出性能を評価する必要がある．また，最適な反復回数は対象回路の規模や生成 HT の多様性によって異なる可能性があり，本評価では 2 回目の反復後に最大 F 値 0.9779 を示したものの，大規模回路ではターゲット候補が増えるため，より多くの反復が必要となる可能性がある．今後は，検出性能の収束と生成 HT の多様性に基づく反復終了基準を検討する．

第二に，生成 HT の検出回避性能には，LLM の安全制約に起因する限界がある．検出器による識別の回避を直接目的とした指示は LLM ガードレールにより制限される場合があるが，プロンプトの最適化により検出回避性能をさらに高められる可能性がある．また，LLM ごとに生成される HT の構造的特徴や検出回避性能が異なる可能性があるため，複数の LLM による比較や，世代ごとに異なる LLM を用いる構成を検討し，生成 HT の多様性および検出器の汎化性能への影響を評価する必要がある．

5.　おわりに

本稿では，LLM による HT 生成とグラフ学習を用いた HT 検出器の強化を反復する，ゲートレベルネットリストを対象としたパープルチームフレームワークを提案した．構文検査および論理等価性検証を通過し，世代ごとに異なる構造とターゲットを持つ HT を生成できることを確認した．生成 HT は既知 HT のみで学習した検出器に対して低い TPR を示した一方，これらを学習データに追加することで，F 値は 0.9226 から最大 0.9779 に向上し，既存のグラフ学習型 HT 検出手法の報告値を上回った．さらに，学習に含まれない世代の HT に対しても高い検出性能を示し，生成 HT を用いた再学習が未知 HT に対する汎化性能の向上に有効であることを確認した．

今後は，より大規模な回路への適用に加え，プロンプトの改善による検出回避性能の向上と，複数の LLM を用いた生成 HT の比較を検討する．

謝辞

本研究は一部，JSPS 科研費（25K03096）の助成を受けたものである．

参考文献

[1] S. Rekhi, K. Amberiadis, A. Akib, and A. Srivastava, “Analyzing collusion threats in the semiconductor supply chain,” *National Institute of Standards and Technology, Gaithersburg, MD, NIST Cybersecurity White Paper*, 2025.

[2] J. Cruz, and J. Hamlet, “A survey on the design, detection, and prevention of pre-silicon hardware trojans,” *IEEe Access*, 2025.

[3] M. Tehranipoor, and F. Koushanfar, “A survey of hardware trojan taxonomy and detection,” *IEEE design & test of computers*, vol. 27, no. 1, pp. 10–25, 2010.

© 2026 Information Processing Society of Japan

[4] R. Mukherjee, S. R. Rajendran, and R. S. Chakraborty, "A comprehensive survey of physical and logic testing techniques for hardware trojan detection and prevention," *Journal of Cryptographic Engineering*, vol. 12, no. 4, pp. 495–522, 2022.

[5] K. Hasegawa, K. Yamashita, S. Hidano, K. Fukushima, K. Hashimoto, and N. Togawa, "Node-wise hardware trojan detection based on graph learning," *IEEE Transactions on Computers*, vol. 74, no. 3, pp. 749–761, 2023.

[6] S. Wang, Y. Cui, S. Yu, C. Gu, C. Wang, and W. Liu, "Gatrojan: An efficient gate-level hardware trojan detection approach using graph attention networks," in *2024 IEEE 37th International System-on-Chip Conference (SOCC)*. IEEE, 2024, pp. 1–6.

[7] P. Ma, J. Li, H. Liu, J. Shi, S. Zhang, W. Pan, and Y. Hao, "Hardware trojan detection methods for gate-level netlists based on graph neural networks," *IEEE Transactions on Computers*, vol. 74, no. 5, pp. 1470–1481, 2025.

[8] D. Cheng, C. Dong, W. He, Z. Chen, and Y. Xu, "Gnn4gate: A bi-directional graph neural network for gate-level hardware trojan detection," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, pp. 1315–1320.

[9] R. Yasaei, L. Chen, S.-Y. Yu, and M. A. Al Faruque, "Hardware trojan detection using graph neural networks," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 1, pp. 25–38, 2025.

[10] Z. Li, W. Cheng, H. Tang, and Y. Wang, "Great: Global representation and edge-attention for hardware trojan detection," in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, 2025, pp. 233–245.

[11] S. Yoshimi, Y. Ikegami, and N. Togawa, "Hardware-trojan detection using graphgps," in *2026 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 2026, pp. 1–2.

[12] "Trust-hub," https://trust-hub.org/, accessed: 2026-07-22.

[13] CAD for Assurance, "Synthetic trojan inserted asic benchmarks (trit-tc)," https://cadforassurance.org/benchmarks/synthetic-trojan-inserted-asic-benchmarks/, accessed: 2026-07-22.

[14] S. Sreekumar, Z. Wang, A. Saha, W. Xiao, M. Shao, M. Shafique, O. Sinanoglu, R. Karri, and J. Knechtel, "Trojangym: A detector-in-the-loop llm for adaptive rtl hardware trojan insertion," *arXiv preprint arXiv:2601.17178*, 2026.

[15] M. O. Faruque, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Unleashing ghost: An llm-powered framework for automated hardware trojan design," *Electronics*, vol. 14, no. 23, p. 4745, 2025.

[16] A. Sarihi, P. Jamieson, A. Patooghy, and A.-H. A. Badawy, "Trojanforge: Generating adversarial hardware trojan examples using reinforcement learning," in *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*, 2024, pp. 1–7.

[17] G. Kumar, A. H. Shaik, A. Riaz, Y. Prasad, and S. Ahlawat, "Compatibility graph assisted automatic hardware trojan insertion framework," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.

[18] A. Waksman, M. Suozzo, and S. Sethumadhavan, "Fanci: identification of stealthy malicious logic using boolean functional analysis," in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 697–708.

[19] R. S. Chakraborty, F. Wolff, S. Paul, C. Papachristou, and S. Bhunia, "Mero: A statistical approach for hardware trojan detection," in *Cryptographic Hardware and Embedded Systems - CHES 2009*. Springer Berlin Heidelberg, 2009, pp. 396–410.

[20] W. Hu, B. Mao, J. Oberg, and R. Kastner, "Detecting hardware trojans with gate-level information-flow tracking," *Computer*, vol. 49, no. 8, pp. 44–52, 2016.

[21] K. Hasegawa, M. Oya, M. Yanagisawa, and N. Togawa, "Hardware trojans classification for gate-level netlists based on machine learning," in *2016 IEEE 22nd International Symposium on On-Line Testing and Robust System Design (IOLTS)*. IEEE, 2016, pp. 203–206.

[22] K. Hasegawa, M. Yanagisawa, and N. Togawa, "Trojan-net feature extraction and its application to hardware-trojan detection for gate-level netlists using random forest," *IEICE Transactions on Fundamentals*, vol. 100, no. 12, pp. 2857–2868, 2017.

[23] C. Dong, Y. Liu, J. Chen, X. Liu, W. Guo, and Y. Chen, "An unsupervised detection approach for hardware trojans," *IEEE Access*, vol. 8, pp. 158 169–158 183, 2020.

[24] S. Yoshimi, Y. Ikegami, R. Negishi, K. Hisafuru, and N. Togawa, "Evaluation of hardware-trojan detection by ensemble learning model for circuits inserted with a trojan by an automated framework," in *2025 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 2025, pp. 1–4.

[25] L. Rampášek, M. Galkin, V. P. Dwivedi, A. T. Luu, G. Wolf, and D. Beaini, "Recipe for a general, powerful, scalable graph transformer," *Advances in Neural Information Processing Systems*, vol. 35, pp. 14 501–14 515, 2022.

[26] S. Brin, and L. Page, "The anatomy of a large-scale hypertextual web search engine," *Computer networks and ISDN systems*, vol. 30, no. 1-7, pp. 107–117, 1998.

[27] L. C. Freeman, "A set of measures of centrality based on betweenness," *Sociometry*, pp. 35–41, 1977.

[28] S. Williams, and M. Baxter, "Icarus verilog: open-source verilog more than a year later," *Linux Journal*, vol. 2002, no. 99, p. 3, 2002.

[29] Cadence Design Systems, Inc., "Conformal Equivalence Checker," Product Datasheet, 2023, accessed: 2026-07-26. [Online]. Available: https://www.cadence.com/en_US/home/resources/datasheets/conformal-equivalence-checker-ds.html

[30] A. Sanfeliu, and K.-S. Fu, "A distance measure between attributed relational graphs for pattern recognition," *IEEE transactions on systems, man, and cybernetics*, no. 3, pp. 353–362, 1983.

[31] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, "Weisfeiler-lehman graph kernels." *Journal of Machine Learning Research*, vol. 12, no. 9, 2011.

[32] Toshiba Information Systems (Japan) Corp., "The hardware trojan detection service," [Online]. Available: https://www.tjsys.co.jp/english/lsi/htfinder/.

© 2026 Information Processing Society of Japan

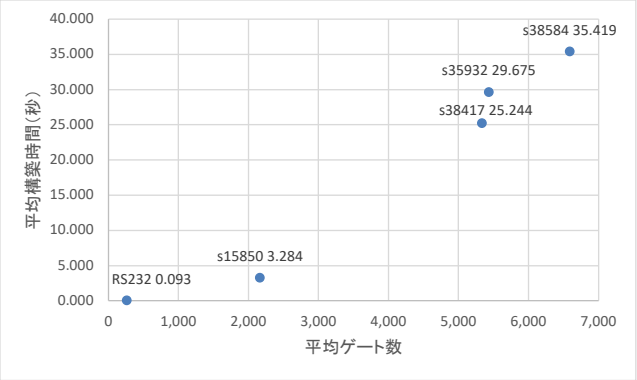


図 A·1　回路規模と 1 回路当たりの平均データ構築時間の関係.

表 A·1　学習データ構成ごとのデータ数と平均学習時間.

学習データ	合計データ数	平均学習時間 (h)
og	17	0.53
og+gen1	123	4.05
og+gen2	54	1.77
og+gen1+gen2	160	5.65
og+gen3	66	2.14
og+gen1+gen2+gen3	209	7.05

付　　録

A.1　実行時間と API コスト

本付録では，提案フレームワークにおける HT 検出器の学習・評価時間と，HT 生成に要する API 利用コストを示す． 2.2 節で示したように，一般に HT 検出器の処理は，データセット作成，学習，および評価の 3 段階から構成される．図 A·1 に，回路規模と 1 回路当たりの平均データ構築時間の関係を示す．キャッシュを使用しない状態で，特徴量とエンコーディングの計算時間を計測した．平均データ構築時間は回路の平均ゲート数に伴って概ね増加し，最大規模の s38584 では 1 回路当たり 35.42 秒を要した．次に，表 A·1 に，学習データ構成ごとの 1 回のモデル作成の平均学習時間を示す．学習回路数の増加に伴って学習時間も概ね増加し，209 回路から構成される og+gen 1+gen 2+gen 3 では，1 回の学習に 208 個の学習データを使用することになり，1000 エポックの学習に平均 7.05 時間を要した．評価時間はすべての回路で 1 回当たり 0.3 秒未満であった．

次に，HT 生成に要する API 利用コストを示す．本稿では，LLM として `gemini-3-flash-preview` を使用したが，HT 生成プラン 1 件当たりの API 利用コストは約$0.01 であった．

© 2026 Information Processing Society of Japan